Study Guide – Exam 3

Distributed & Cloud Computing (CSC-4770-001)

Prepared for: Celia Bedilia

Fall 2025

November 14, 2025

Contents

1 Out-of-Band Communication

1.1 Issue

Can a server be interrupted after accepting or processing a service request?

1.2 Methods

1.2.1 Separate Port for Urgent Data

- Use a dedicated thread/process for urgent messages; current request on hold.
- Requires OS support for priority-based scheduling.

1.2.2 Transport Layer Support

- TCP can carry urgent messages on the same connection, intercepted via OS signaling.

1.3 Incident Response Context

- Out-of-band (OOB) communication uses independent, secure channels separate from the primary network (e.g., encrypted messaging apps, satellite phones).
- Benefits: communication continuity during outages or attacks; faster incident resolution; protection of sensitive information; reduces reliance on compromised systems.
- Access controls: strong authentication and multi-factor authentication safeguard OOB channels.

2 Servers and State

2.1 Stateless Servers

- Do not maintain client status after handling a request (no remembered opened files, cache states, or client info).
- Benefits: independence between client and server; reduces state inconsistency risks from crashes.
- Trade-offs: potential performance loss due to lack of anticipatory caching or session reuse.

2.2 Stateful vs Stateless

- Compare based on performance and reliability trade-offs.
- Note: exams often require articulating when stateful design improves user experience vs when stateless enhances scalability.

3 Client/Server and RPC Concepts

3.1 Client/Server Model

- Transient synchronous communication: client blocks waiting for a server reply; server processes requests sequentially.
- Drawbacks: blocking client; immediate failure handling required; not suitable for all applications (e.g., email).

3.2 Message-Oriented Middleware

- Asynchronous persistent communication.
- Messages are queued; sender need not wait for a reply.

- Benefits: fault tolerance and scalability enhancements.

3.3 RPC (Remote Procedure Call)

- Basic flow: stubs on client/server pack/unpack messages; OS handles transmission; remote execution occurs.
- Parameter passing: copying in/out semantics; data marshaling; encoding to handle machine differences.
- Access transparency: limited due to difficulty passing references and global data.
- Variants:
  - Asynchronous RPC: client continues without waiting for response.
  - Multicast RPC: RPC to multiple servers for load balancing and fault tolerance.

4 Threads and Processes (Exam Context)

4.1 Threads

Minimal software processors executing instructions; lightweight relative to processes.

4.2 Processes

Context for one or more threads; heavier due to distinct memory and resources.

4.3 Threading Models

- User-level threads: managed by user-space library; OS unaware; fast but blocking can affect the whole process.
- Kernel-level threads: scheduled by OS; higher overhead but finer-grained concurrency.
- Hybrid models: mix of user and kernel threads.

4.4 Multithreading in Clients/Servers

Goals: hide network latency; allow parallel requests; improve responsiveness (e.g., GUI apps, servers with dispatcher/worker patterns).

5 Virtualization (Foundational Concepts)

5.1 Purpose and Benefits

Portability, isolation, migration.

5.2 VMM Types

- Process VM: runs as an application providing a virtualized environment.
- Native VMM: runs directly on hardware with multiple OS instances; lower overhead.
- Hosted VMM: runs on a host OS; leverages host services; typically higher overhead.

5.3 Instruction Types

Sensitive vs. Privileged instructions; VMM handles sensitive instructions securely.

5.4 Paravirtualization and Containers

- Paravirtualization: modified guest OS for efficient host communication.
- Containers: OS-level virtualization; lightweight isolation using namespaces and cgroups; faster deployment but less isolation than full VMs.

6 Server Design and Object/Distributed Systems

6.1 Stateful vs Stateless Objects

- Stateful objects retain data across invocations; affects concurrency and scalability.
- Stateless objects process each request independently; easier to scale.

6.2 Activation Policies for Distributed Objects

Single-threaded, per-thread, thread pool models. Understand differences in concurrency, resource use, and throughput.

6.3 Server Clusters and CDNs

Three-tier architectures; dispatching and TCP handoff. CDNs (e.g., Akamai) for content delivery, caching, and performance.

7 Code Migration and Mobility (Migration Focus)

7.1 Code Migration and Mobility Models

Weak mobility: code and data move between machines. Strong mobility: code, data, and execution state move; supports mid-execution migration.

7.2 VM Migration Techniques

Push: proactively migrate VM state. Stop-and-copy: halt VM to transfer state (downtime impact). Pull-on-demand: fetches needed state dynamically. Performance considerations: downtime minimization and network overhead.

8 Layered Communication Models and OSI (Ch.4 Context)

8.1 OSI Model Layers

Physical, Data Link, Network, Transport, Session, Presentation, Application.

8.2 Low-Level Layer Roles

Physical: bit transmission. Data Link: frames and error control. Network: packet routing.

8.3 Transport Layer Roles

TCP: reliable, stream-based delivery. UDP: unreliable, datagram-based delivery.

8.4 Middleware Layer Services

Marshaling, naming, security, scaling.

8.5 Socket Programming and ZeroMQ

Berkeley Socket API: socket, bind, listen, accept, connect, send, recv, close. ZeroMQ patterns: Request-Reply, Publish-Subscribe, Pipeline.

9 Practice Notes

- Use this outline to study: focus on definitions, trade-offs, and example scenarios.
- Draw diagrams to illustrate client/server architectures, VM/container layers, and

OSI mappings.

- Practice converting between transient/persistent and synchronous/asynchronous communication in MoM and RPC contexts.